INDITEX UDC

Cátedra de IA en Algoritmos Verdes

MICROCREDENCIAL ALGORITMOS VERDES PARA LA IA

DISEÑO E IMPLEMENTACIÓN

Tema: Natural language processing

Sesión 1: Green NLP Today

Ponente: Olga Zamaraeva

citic

UNIVERSIDADE DA CORUÑA

Financiado por la Unión Europea NextGenerationEU

GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA

ENIA ESTRATEGIA NACIONAL DE INTELIGENCIA ARTIFICIAL

España | digital 2026

LET'S START WITH A SENTENCE

*Unha **tía miña**, que todo o solucionaba con ditos e refráns, dicía algo así como*

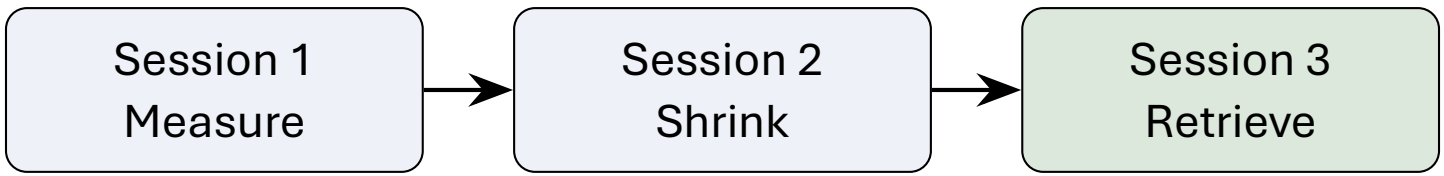
"un é escravo do que di e dono do que cala"

from "Speechwriter" by Paulo Naseiro

MODULE OVERVIEW

Three sessions, one running project

- Session 1: **Measure** the baseline
- Session 2: **Shrink** it — quantization
- Session 3: **Retrieve** on demand

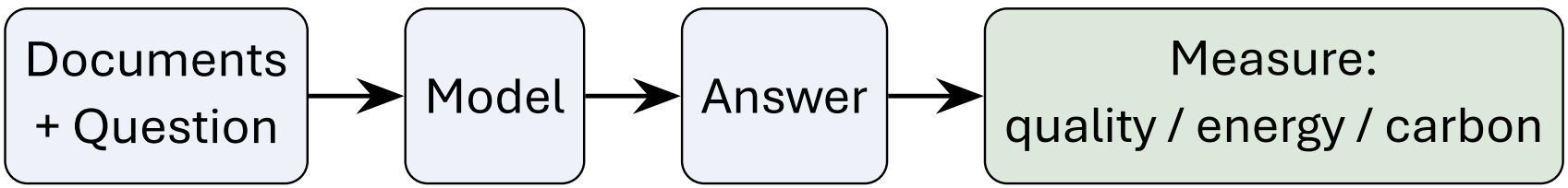


olga.zamaraeva@udc.es • UDC Teams • CITIC, 2nd floor

THE RUNNING PROJECT: GREEN NLP BENCHMARK LAB

One pipeline, one change at a time

- Same pipeline, all three sessions — change **one** thing, re-measure
- Quality vs. energy vs. carbon — your own leaderboard



INDITEX UDC
Cátedra de IA en Algoritmos Verdes

HOW THIS SESSION WORKS

Fewer slides, more interactive pages

- Much of today is spent on **interactive pages**: animations you can step through, change and replay
- The slides introduce each idea; the pages show it working
- Open them on your own laptop or phone, now or later:

olzama.github.io/teaching/GreenAI

- When a slide shows this box, we leave the slides and go to the page:

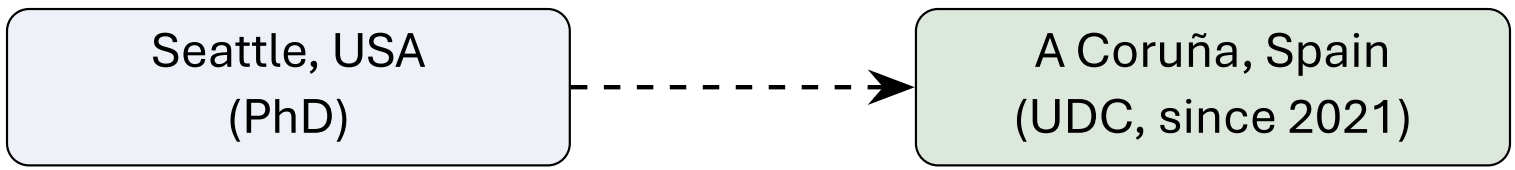
► SWITCH TO THE PAGE **N-grams, step by step**

The page and the sections we use. Then we come back to the slides.

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

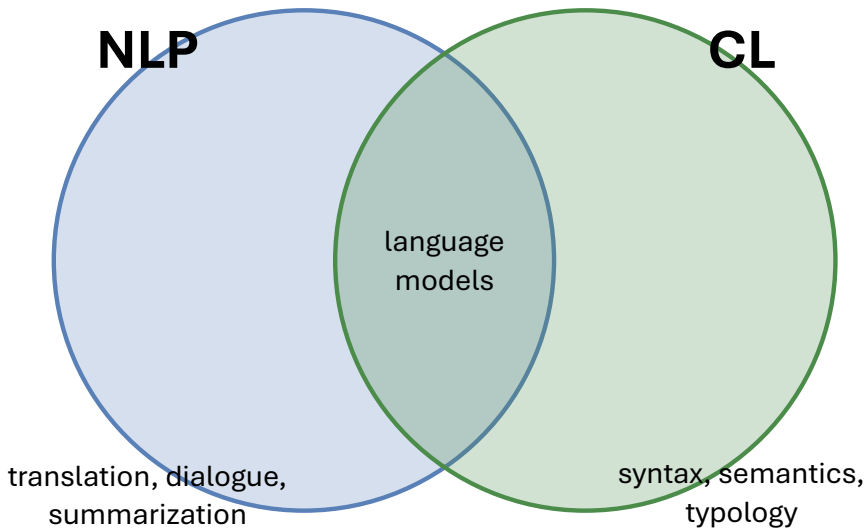
INSTRUCTOR'S PROFILE

- Computational linguist — syntax and machine translation



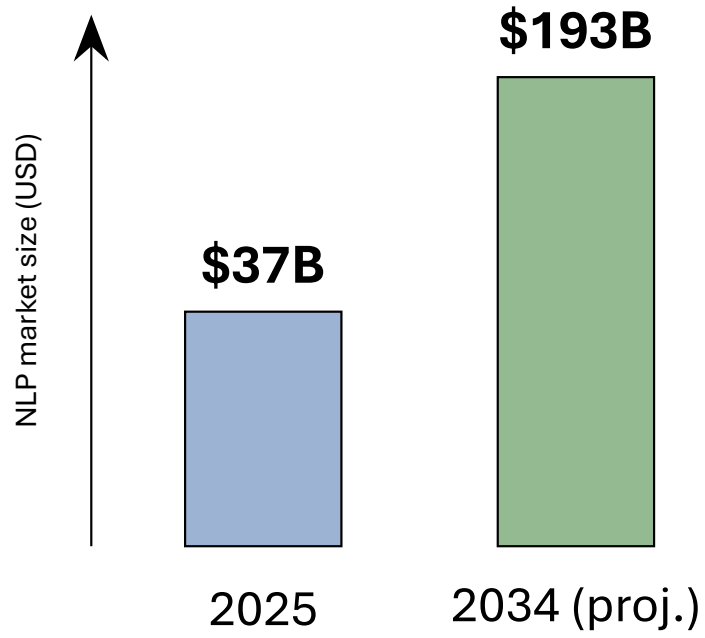
NATURAL LANGUAGE PROCESSING & COMPUTATIONAL LINGUISTICS

- NLP: building systems that *do* things with language
- CL: using computation to *study* language



INDITEX UDC
Cátedra de IA en Algoritmos Verdes

HOW BIG IS NLP, REALLY?



- ∼20% annual growth — but estimates vary widely across firms (as low as $25B for the same year, elsewhere)

Source: Fortune Business Insights, 2025

WHERE NLP SHOWS UP

Tech & Telecom

Banking & Finance

Healthcare (fastest-growing)

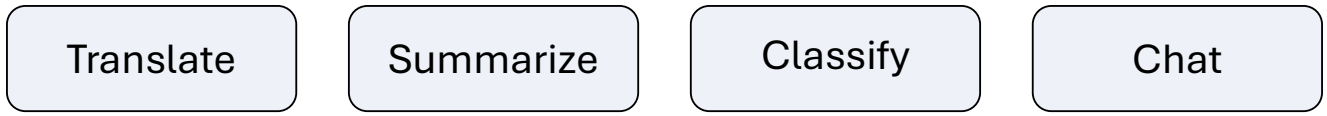
Chatbots

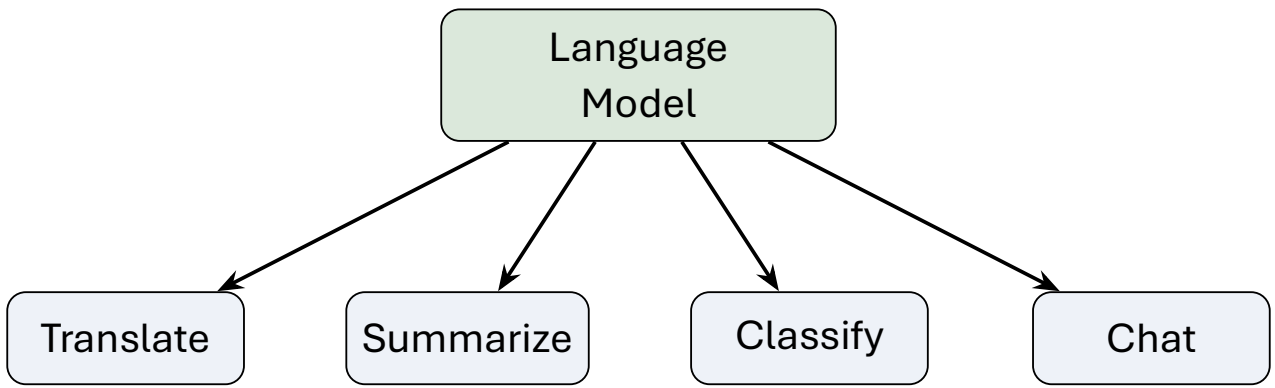
Translation

Speech & Sentiment

ONE ENGINE NOW RUNS MANY TASKS

separate models, one per task

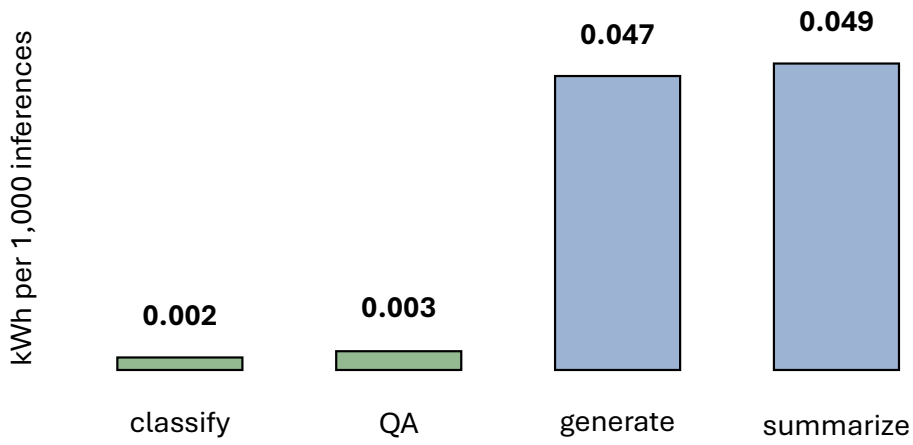




one model, prompted — this is where “NLP” and “AI” blur

Source: Radford et al., “Language Models are Unsupervised Multitask Learners,” 2019; Brown et al., “Language Models are Few-Shot Learners,” NeurIPS 2020

NLP TASKS: WHAT THEY ACTUALLY COST

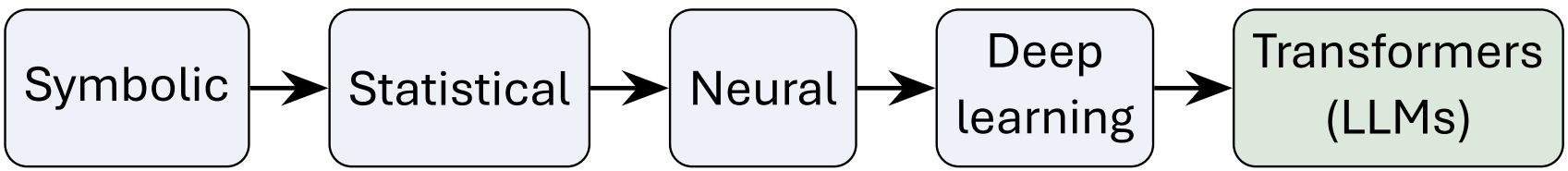


- Discriminative tasks (classify, answer) are cheap; generative tasks (write, summarize) cost 15–20× more
- Image generation: 2.9 kWh per 1,000 — about 1,450× the cheapest text task

Source: Luccioni, Jernite & Strubell, FAccT 2024 (arXiv:2311.16863), Table 2

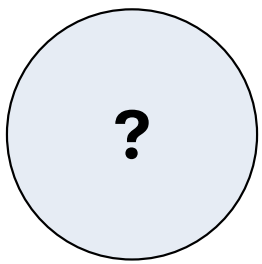
HOW WE GOT HERE

- Symbolic: rules written by hand (1950s–80s) → Statistical: counting from real text (1990s–2000s) → Neural: learned representations, more data and compute needed (2010s+) → Transformers: attention-based (2017+)
- Each shift trades hand-engineering for more data and compute — directly relevant to a course about the cost of that compute

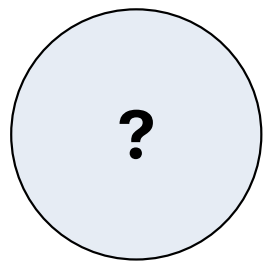


BEFORE WE START: GUESS THE FOOTPRINT

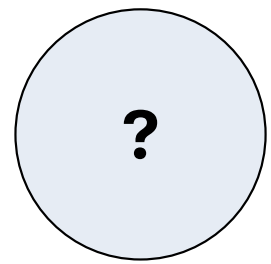
1. How many metric tons of CO_2eq did training GPT-3 emit?
2. What share of global electricity do data centers consume today?
3. How many times more carbon does a general-purpose LLM emit per query than a chatbot trained and built just for that task?



training emissions



global electricity



task-specific vs. general-purpose

REVEAL 1: TRAINING A LARGE MODEL

552 t CO_2eq $\approx$

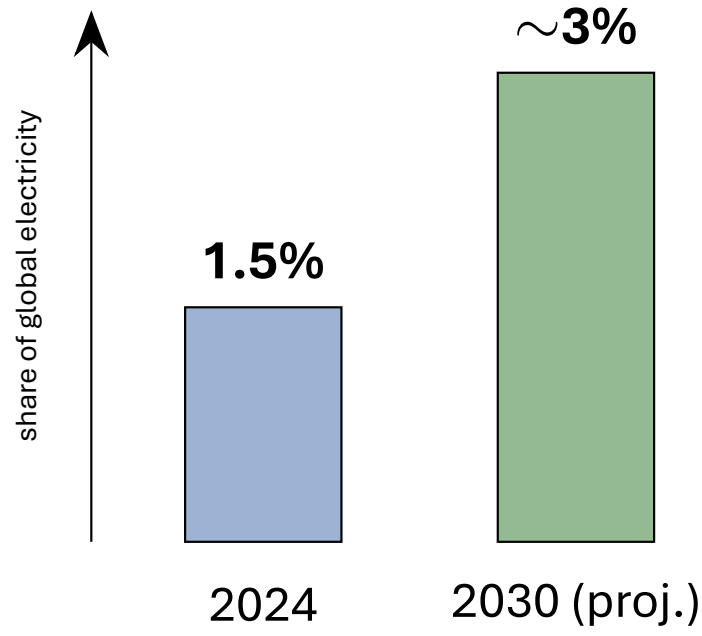


3 $\times$ SF $\leftrightarrow$ NYC round trips

- A *one-time* cost — paid once, by whoever trains the model

Source: Patterson et al., “Carbon Emissions and Large Neural Network Training,” 2021 (arXiv:2104.10350)

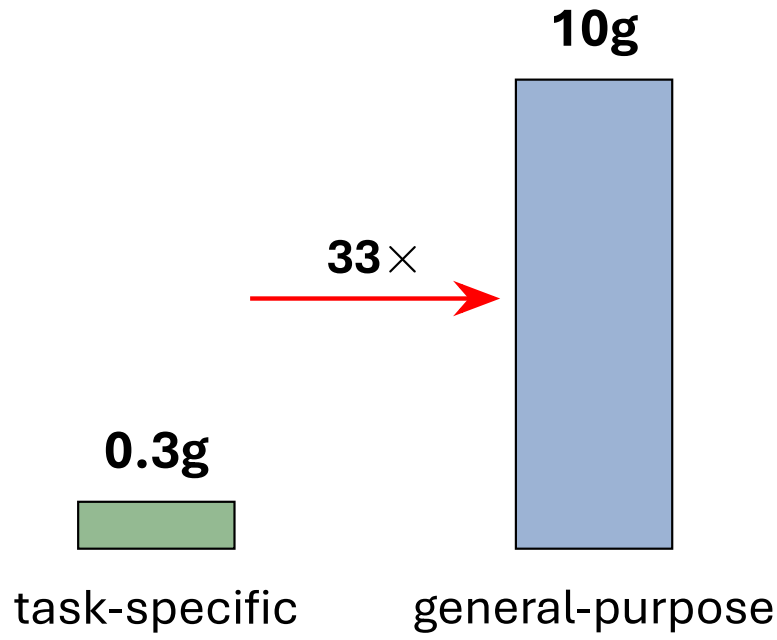
REVEAL 2: DATA CENTERS, GLOBALLY



- Growing 17% in just 2025 — driven mainly by AI

Source: International Energy Agency, “Energy and AI,” 2024–2025

REVEAL 3: GENERAL-PURPOSE LLMS ARE COSTLY



CO_2 eq per 1,000 inferences, same task

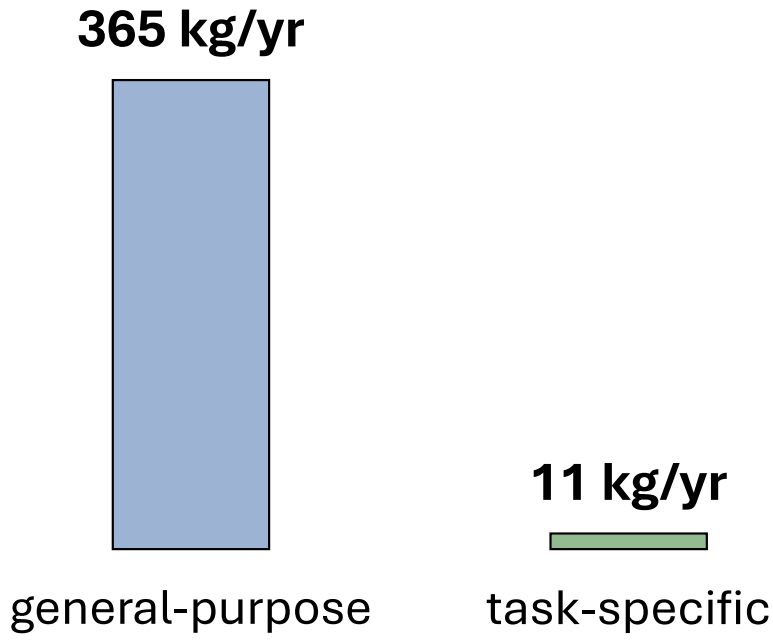
- Same task, 33× apart

Source: Luccioni, Jernite & Strubell, FAccT 2024 (arXiv:2311.16863)

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

EXAMPLE: THE 33×, AT DEPLOYMENT SCALE

- A chatbot answering **100,000 questions/day**, general-purpose LLM API
- Anchor: 10g CO_2eq / 1,000 inferences (Reveal 3)

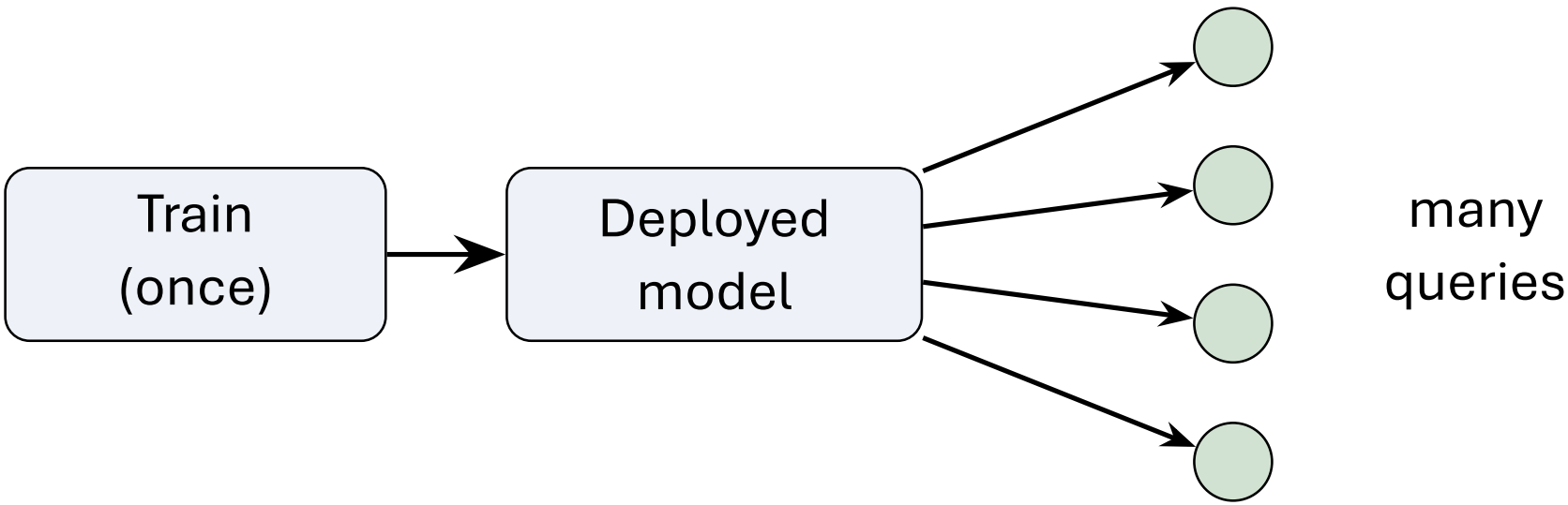


- 100k/day × 10g/1,000 ≈ 365 kg/year — about a month of driving a car
- Swap to task-specific (Reveal 3's 33×) → ≈ 11 kg/year

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

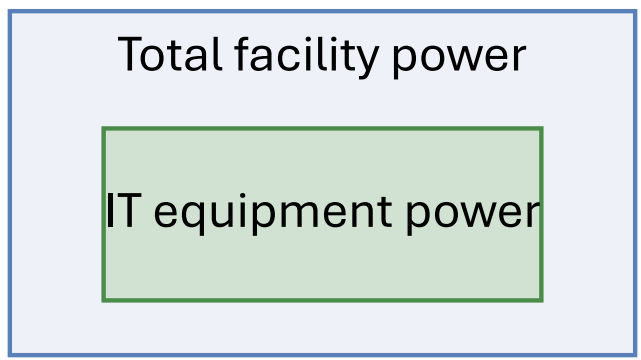
TRAINING VS. INFERENCE

- Train **once**. Infer **millions of times**
- Cumulative inference cost dominates a model's lifetime footprint



VOCABULARY FOR THIS COURSE

- **FLOPs** (floating-point operations): how much computation
- **Carbon intensity**: CO_2eq per unit of electricity — depends on *where/when*
- **PUE** (Power Usage Effectiveness): facility power ÷ IT equipment power
- **Operational** vs. **embodied** carbon: running it vs. building it



PUE = facility power ÷ IT equipment power

CASE STUDY: A FAMOUS NUMBER, REVISITED



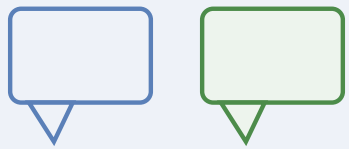
"= 5 cars, lifetime emissions"

- One 2019 paper, number repeated everywhere for years — later shown $\sim 10\times$ too high (actual number 10x smaller)

Source: Strubell et al., ACL 2019

Activity: Class discussion (10 min)

- Why is estimating carbon footprint from a paper so hard?
- What would you need to *trust* a number like this?
- Catchy comparisons: help, or hurt?



DISCUSSION: SOME POSSIBLE ANSWERS

- **Hard because**: hardware, PUE, run time, grid — rarely all reported
- **To trust it**: the full method, ideally reproducible code
- **Catchy comparisons**: memorable — risky if the number is wrong

BACK TO THAT SENTENCE

*Unha **tía miña**, que todo o solucionaba con ditos e refráns, dicía algo así como*

“un é escravo do que di e dono do que cala”

an aunt’s saying, from the novel *Speechwriter* by Paulo Naseiro

- Galician: a **low-resource** language
- Does a huge general model actually handle a case like this, or does it rely on what it has plenty of data for — Spanish, Portuguese, English?
- But… what’s a “model”?

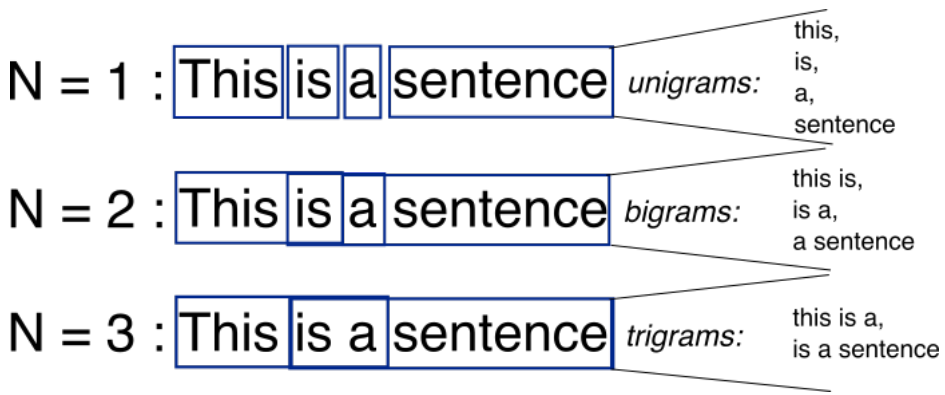
FROM N-GRAMS TO TRANSFORMERS

Each step fixes a problem of the step before

Step	What it adds	What is still missing	Page
1 N-grams	predict the next word by counting	aunt and uncle are unrelated words	N-grams
2 Embeddings	a learned vector per word: similar words get similar vectors	vectors for words, not for sentences	Skip-gram
3 Encoder–decoder	read a sentence into one vector, write the translation from it	one vector for a whole sentence: early words fade	Neural translation
4 Attention	the decoder looks back at every source word	the source is still read one word after another	Attention 1–3
5 Self-attention	every word looks at every word of its own sentence, all at once	one round of looking	Attention 4–5
6 Transformers	many blocks of self-attention, stacked: today's LLMs	cost grows with size and length	slides

WHAT IS A LANGUAGE MODEL?

- Predicts the next word — a full probability distribution over the vocabulary
- N-grams: the simplest version



Credit: Jurafsky & Martin, *Speech and Language Processing*

N-GRAMS: A WORKED EXAMPLE

my aunt stayed quiet • *my aunt spoke wisely* • *my uncle stayed quiet*

$$P(w_i \mid w_{i-1}) = \frac{\text{Count}(w_{i-1}\, w_i)}{\text{Count}(w_{i-1})}$$

- $P(\texttt{aunt} \mid \texttt{my}) = 2/3 \approx 0.67 \qquad P(\texttt{uncle} \mid \texttt{my}) = 1/3 \approx 0.33$
- Counting, then dividing — that's the whole model
- A Galician model has far fewer sentences to count from than a Spanish or English one — noisier estimates

▶ SWITCH TO THE PAGE **N-grams, step by step**

Sections 4–6: count and divide, generate text, and why one unseen pair gives a sentence probability 0.

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

Activity: N-grams Playground (∼15 min, individual)

- Open `01b_ngrams_playground.ipynb` — CPU only, no GPU needed
- Part A: rebuild this exact bigram model yourself, then generate text with it
- Part B: use it to fix the word order of a broken word-for-word translation — a preview of Notebook 1's "statistical" approach



FROM COUNTS TO VECTORS

- Count co-occurrences instead of sequences → a vector per word

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	1	8	15
soldier	2	2	12	36
fool	37	58	1	5
clown	6	117	0	0

COMPARING VECTORS: COSINE SIMILARITY

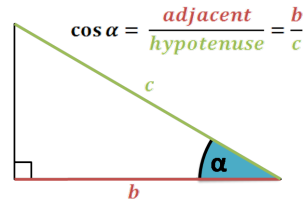
$$\cos(u, v) = \frac{u \cdot v}{\|u\| \, \|v\|}$$

- Toy example: aunt $= (4, 2)$, uncle $= (3, 3)$, car $= (1, 4)$
- $\cos(\text{aunt}, \text{uncle}) \approx 0.95$ $\quad$ $\cos(\text{aunt}, \text{car}) \approx 0.65$
- Dividing by length makes this about *direction* alone

$$\text{dot-product}(\vec{v}, \vec{w}) = \vec{v} \cdot \vec{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \ldots + v_N w_N$$

$$\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| \cos\theta$$

$$\frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|} = \cos\theta$$



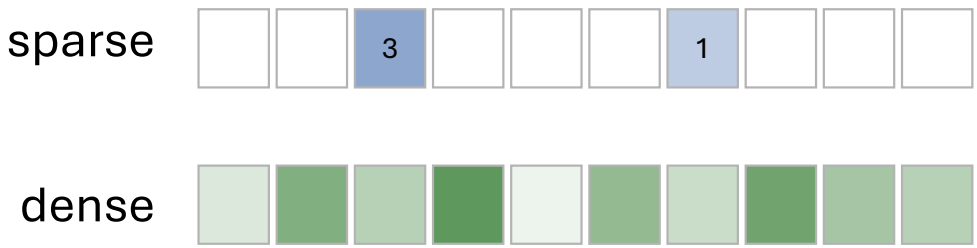
► **SWITCH TO THE PAGE** **N-grams, step by step**

Section 1: four words counted in four Shakespeare plays.
Plays and words become vectors; cosine on every pair.

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

FROM WORDS TO VECTORS: EMBEDDINGS

- **Learned**, **dense** vectors — every dimension has a value
- Similar contexts → similar vectors — same cosine idea, learned this time



same word, two representations



Credit: jalammar.github.io/illustrated-transformer/

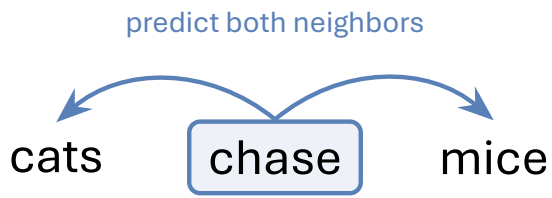
HOW EMBEDDINGS ARE ACTUALLY LEARNED

Skip-gram (Mikolov et al., 2013): predict the neighbors

A tiny corpus

cats chase mice
dogs chase balls
cats eat fish
dogs eat meat

Window = 1



16 training pairs

input → word to predict

cats	→ chase	cats	→ eat
chase	→ cats	eat	→ cats
chase	→ mice	eat	→ fish
mice	→ chase	fish	→ eat
dogs	→ chase	dogs	→ eat
chase	→ dogs	eat	→ dogs
chase	→ balls	eat	→ meat
balls	→ chase	meat	→ eat

- The task: given *cats*, give high probability to *chase* and *eat*
- The task never mentions word vectors. The vectors appear because the model needs them to make predictions.

INSIDE SKIP-GRAM: WHAT GETS MULTIPLIED

Two tables of parameters. Both start random.

- p: the **embedding table**, one row per word. A **one-hot** vector (all 0, one 1) times p copies out one row, h:

$$\begin{pmatrix} 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0.1 & 0.9 \\ 0.3 & 0.2 \\ 0.7 & 0.5 \\ 0.4 & 0.8 \end{pmatrix} = \begin{pmatrix} 0.7 & 0.5 \end{pmatrix}$$

- p': one column per word. $h \times p'$ gives one **score** per word; **softmax** turns scores into probabilities that sum to 1
- Training nudges p and p': the true neighbor gets more probability

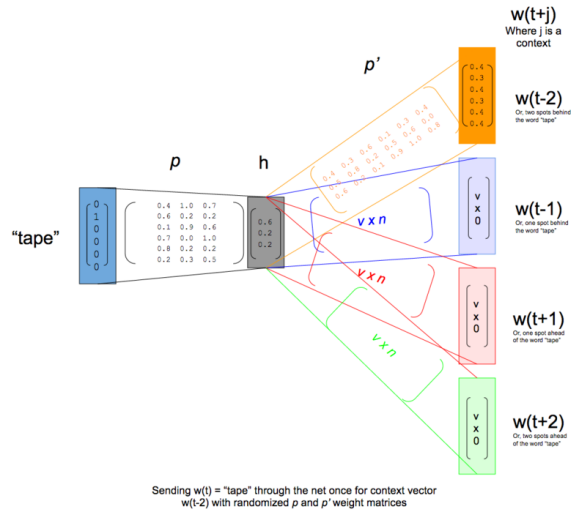
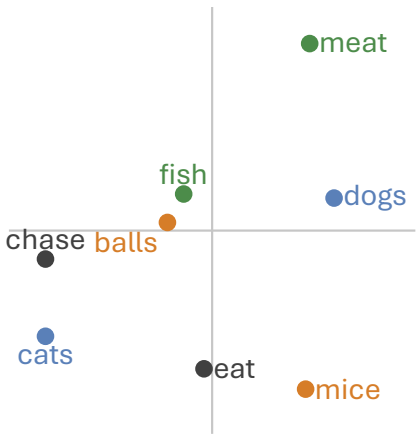


Illustration: district-data-labs (Medium)

▶ SWITCH TO THE PAGE **Skip-gram, step by step**

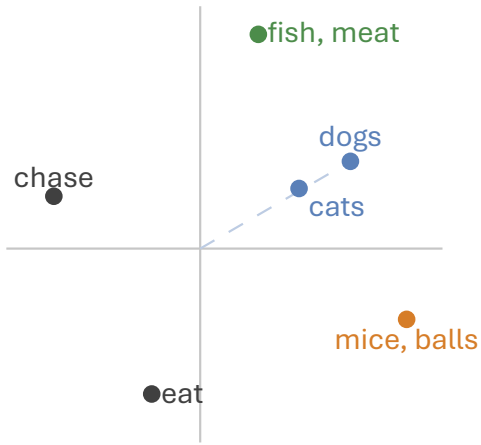
Top: the multiplication for one pair, then for all words at once. Then train our corpus: Step, One pass, Play.

THE SAME TABLE, REDRAWN DURING TRAINING



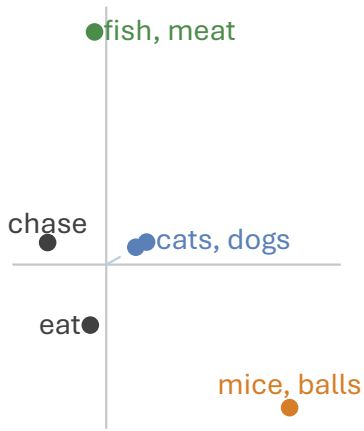
start

axes: -0.5 to 0.5



100 passes

axes: -2 to 2.5



3000 passes

axes: -2 to 5

- One pass = all 16 training pairs once
- Same color $\rightarrow$ same direction from the origin. Nobody told the model which words belong together.

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

SIMILAR COMPANY, SIMILAR VECTORS

The company each word keeps

cats	chase, eat
dogs	chase, eat
mice	chase
balls	chase
fish	eat
meat	eat

The table after training

cats	0.63	0.37
dogs	0.86	0.48
mice	3.89	-3.15
balls	3.93	-2.94
fish	-0.34	5.00
meat	-0.16	4.91

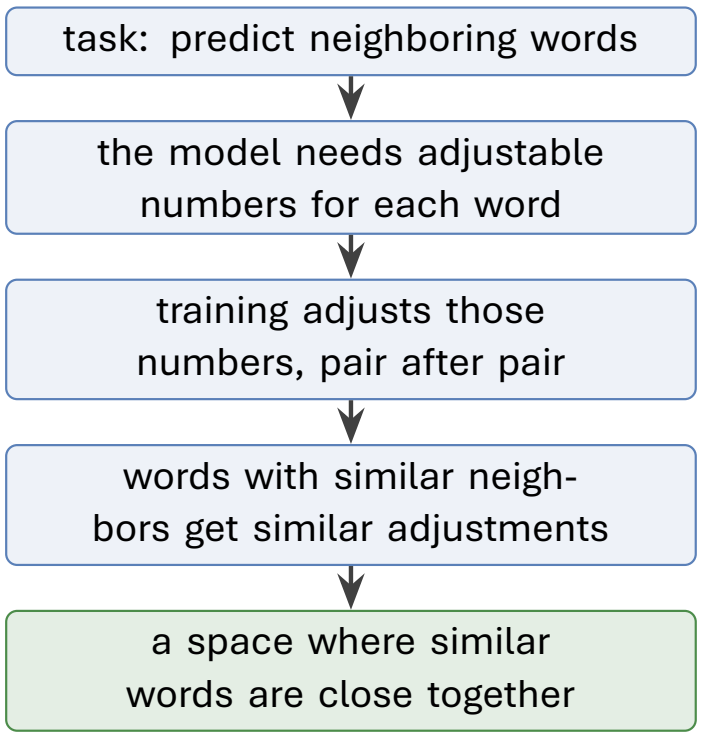
- Same neighbors $\rightarrow$ same nudges, pair after pair $\rightarrow$ same direction
- $\cos(\text{cats}, \text{dogs}) = 1.00 \qquad \cos(\text{cats}, \text{fish}) = 0.44$
- Task solved: $P(\text{chase} \mid \text{cats})\ 0.11 \rightarrow 0.47,\ P(\text{eat} \mid \text{cats})\ 0.12 \rightarrow 0.52$

"You shall know a word by the company it keeps." — J. R. Firth (1957)

▶ SWITCH TO THE PAGE **Skip-gram, step by step**

"Which words are similar?": one-hot vectors vs. trained vectors with many or with 2 dimensions, on corpora of up to 92 words.

WHAT WE KEEP: THE EMBEDDING TABLE



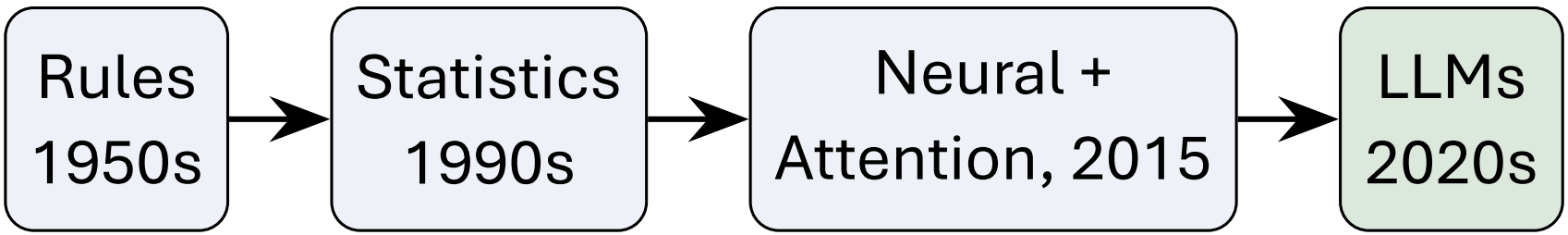
- After training, discard the prediction part and **keep the table**. Its rows are the **word embeddings**.
- Embeddings = the parameters the model uses to solve its prediction task
- Rare word → few pairs → few nudges → poorly placed vector
- Less Galician text online → weaker embeddings for many Galician words

Activity: Embeddings Playground (∼15–20 min, individual)

- Open `01c_embeddings_playground.ipynb` — CPU only, no GPU needed
- Part A: real word embeddings, real cosine similarity, a 2-D map of related words
- Part B: do Galician, Spanish, and Portuguese words for “aunt” really look distinct to a multilingual model?



MACHINE TRANSLATION: A VERY SHORT HISTORY

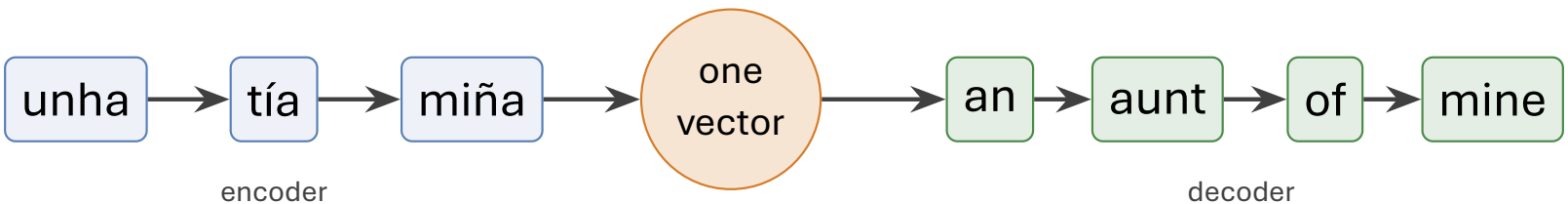


- One of AI's oldest goals — first public demo: 1954 (Georgetown–IBM)
- 1966: the ALPAC report cuts U.S. funding for over a decade
- 2006: Google Translate, built on statistical MT
- Galician is living through this same arc right now

Source: Hutchins, "Machine Translation: A Concise History," 2007

NEURAL TRANSLATION: AN ENCODER AND A DECODER

- Every word is a vector, learned from translated examples, like skip-gram's embeddings
- **Encoder**: reads the Galician words in order into **one vector**, the summary
- **Decoder**: starts from the summary and writes English one word at a time — each word chosen the way skip-gram chose neighbors



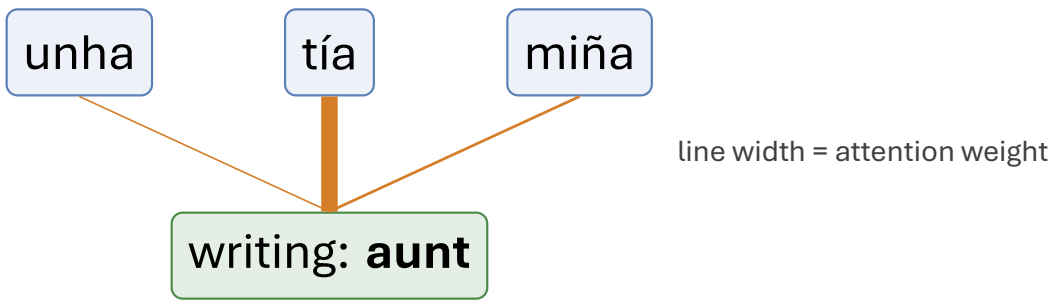
Sutskever, Vinyals & Le, NeurIPS 2014 (arXiv:1409.3215)

► SWITCH TO THE PAGE Neural translation, step by step

Section 1: training on 16 example phrases. Section 2: the encoder's summary. Section 3: the decoder writes, word by word.

ATTENTION WAS BORN IN MACHINE TRANSLATION

- The bottleneck: the whole sentence squeezed into **one vector**
- Bahdanau, Cho & Bengio (2015): the decoder **looks back** at every source word and chooses new weights for every word it writes — **attention**
- Invented for translation, years before today's LLMs



Source: Bahdanau, Cho & Bengio, ICLR 2015 (arXiv:1409.0473)

► SWITCH TO THE PAGE **Attention, step by step**

Section 1: one vector vs. looking back at every word, on *tía miña* and on the whole sentence.

TWO TRAINED MODELS: WITHOUT AND WITH ATTENTION

- Same training data (5,000 sentences built from templates around ours), same size, same training — the only difference is attention
- Tested on 500 sentences that were not in the training data; our sentence was not in it either

vector size	without attention	with attention
8	0	237
16	226	500
64	500	500

whole sentences exactly right, out of 500

Our sentence, 8 dimensions, without attention: “A friend of mine, who fixed everything with stories and jokes, used to repeat something like ‘one is a slave to what one keeps quiet’ ”

► **SWITCH TO THE PAGE** **Attention, step by step**

Section 2: both models write our sentence word by word; the decoder’s state, the scores and the weights at every step.

THE SCORE AND THE WEIGHT, STEP BY STEP

Writing "aunt", 16 dimensions. h: the decoder's new state. $\mathrm{e}_{\text{tía}}$: the encoder's vector at tía.

$$\text{score}(\text{tía}) = \mathrm{h} \cdot \mathrm{e}_{\text{tía}}$$

THE SCORE AND THE WEIGHT, STEP BY STEP

Writing "aunt", 16 dimensions. h: the decoder's new state. $\mathrm{e}_{\text{tía}}$: the encoder's vector at tía.

$$\begin{aligned}\text{score}(\text{tía}) &= \mathbf{h} \cdot \mathbf{e}_{\text{tía}} \\ &= h_1 e_1 + h_2 e_2 + h_3 e_3 + \cdots + h_{16} e_{16}\end{aligned}$$

THE SCORE AND THE WEIGHT, STEP BY STEP

Writing "aunt", 16 dimensions. h: the decoder's new state. $\mathrm{e}_{\text{tía}}$: the encoder's vector at tía.

$$\begin{aligned}\text{score}(\text{tía}) &= \mathbf{h} \cdot \mathrm{e}_{\text{tía}} \\ &= h_1 e_1 + h_2 e_2 + h_3 e_3 + \cdots + h_{16} e_{16} \\ &= 0.91 \cdot (-1.00) + 1.00 \cdot 0.00 + 0.97 \cdot 1.00 + \cdots + 0.99 \cdot 0.98\end{aligned}$$

THE SCORE AND THE WEIGHT, STEP BY STEP

Writing "aunt", 16 dimensions. h: the decoder's new state. $\mathrm{e}_{\text{tía}}$: the encoder's vector at tía.

$$
\begin{aligned}
\text{score}(\text{tía}) &= \mathbf{h} \cdot \mathbf{e}_{\text{tía}} \\
&= h_1 e_1 + h_2 e_2 + h_3 e_3 + \cdots + h_{16} e_{16} \\
&= 0.91 \cdot (-1.00) + 1.00 \cdot 0.00 + 0.97 \cdot 1.00 + \cdots + 0.99 \cdot 0.98 \\
&= -0.91 + 0.00 + 0.97 + \cdots + 0.97
\end{aligned}
$$

THE SCORE AND THE WEIGHT, STEP BY STEP

Writing "aunt", 16 dimensions. h: the decoder's new state. $\mathrm{e}_{\text{tía}}$: the encoder's vector at tía.

$$
\begin{aligned}
\text{score}(\text{tía}) &= \mathbf{h} \cdot \mathbf{e}_{\text{tía}} \\
&= h_1 e_1 + h_2 e_2 + h_3 e_3 + \cdots + h_{16} e_{16} \\
&= 0.91 \cdot (-1.00) + 1.00 \cdot 0.00 + 0.97 \cdot 1.00 + \cdots + 0.99 \cdot 0.98 \\
&= -0.91 + 0.00 + 0.97 + \cdots + 0.97 \; = 8.48
\end{aligned}
$$

THE SCORE AND THE WEIGHT, STEP BY STEP

Writing "aunt", 16 dimensions. h: the decoder's new state. $\mathrm{e}_{\text{tía}}$: the encoder's vector at tía.

$$
\begin{aligned}
\text{score}(\text{tía}) &= \mathbf{h} \cdot \mathrm{e}_{\text{tía}} \\
&= h_1 e_1 + h_2 e_2 + h_3 e_3 + \cdots + h_{16} e_{16} \\
&= 0.91 \cdot (-1.00) + 1.00 \cdot 0.00 + 0.97 \cdot 1.00 + \cdots + 0.99 \cdot 0.98 \\
&= -0.91 + 0.00 + 0.97 + \cdots + 0.97 \; = 8.48 \\
\text{score}(.) = 5.87, \quad \text{score}(\text{miña}) &= 4.91, \quad \text{score}(\text{escravo}) = 4.15, \ \ldots
\end{aligned}
$$

THE SCORE AND THE WEIGHT, STEP BY STEP

Writing "aunt", 16 dimensions. h: the decoder's new state. $\mathrm{e}_{\text{tía}}$: the encoder's vector at tía.

$$
\begin{aligned}
\text{score}(\text{tía}) &= \mathbf{h} \cdot \mathbf{e}_{\text{tía}} \\
&= h_1 e_1 + h_2 e_2 + h_3 e_3 + \cdots + h_{16} e_{16} \\
&= 0.91 \cdot (-1.00) + 1.00 \cdot 0.00 + 0.97 \cdot 1.00 + \cdots + 0.99 \cdot 0.98 \\
&= -0.91 + 0.00 + 0.97 + \cdots + 0.97 \; = 8.48 \\
\text{score}(.) &= 5.87, \quad \text{score}(\text{miña}) = 4.91, \quad \text{score}(\text{escravo}) = 4.15, \ \ldots \\
\text{weight}(\text{tía}) &= \frac{e^{\text{score}(\text{tía})}}{e^{\text{score}(\text{unha})} + e^{\text{score}(\text{tía})} + \ldots}
\end{aligned}
$$

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

THE SCORE AND THE WEIGHT, STEP BY STEP

Writing "aunt", 16 dimensions. h: the decoder's new state. $\mathrm{e}_{\text{tía}}$: the encoder's vector at tía.

$$
\begin{aligned}
\text{score}(\text{tía}) &= \mathrm{h} \cdot \mathrm{e}_{\text{tía}} \\
&= h_1 e_1 + h_2 e_2 + h_3 e_3 + \cdots + h_{16} e_{16} \\
&= 0.91 \cdot (-1.00) + 1.00 \cdot 0.00 + 0.97 \cdot 1.00 + \cdots + 0.99 \cdot 0.98 \\
&= -0.91 + 0.00 + 0.97 + \cdots + 0.97 \; = 8.48 \\
\text{score}(.) &= 5.87, \quad \text{score}(\text{miña}) = 4.91, \quad \text{score}(\text{escravo}) = 4.15, \ \ldots \\
\text{weight}(\text{tía}) &= \frac{e^{\text{score}(\text{tía})}}{e^{\text{score}(\text{unha})} + e^{\text{score}(\text{tía})} + \ldots} \\
&= \frac{e^{8.48}}{e^{8.48} + e^{5.87} + e^{4.91} + e^{4.15} + \ldots}
\end{aligned}
$$

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

THE SCORE AND THE WEIGHT, STEP BY STEP

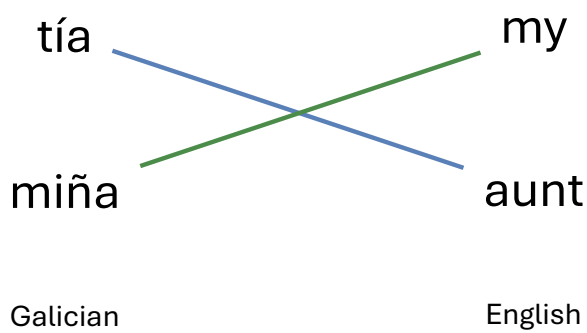
Writing "aunt", 16 dimensions. h: the decoder's new state. $\mathrm{e}_{\text{tía}}$: the encoder's vector at tía.

$$
\begin{aligned}
\text{score}(\text{tía}) &= \mathrm{h} \cdot \mathrm{e}_{\text{tía}} \\
&= h_1 e_1 + h_2 e_2 + h_3 e_3 + \cdots + h_{16} e_{16} \\
&= 0.91 \cdot (-1.00) + 1.00 \cdot 0.00 + 0.97 \cdot 1.00 + \cdots + 0.99 \cdot 0.98 \\
&= -0.91 + 0.00 + 0.97 + \cdots + 0.97 \ = 8.48 \\
\text{score}(.) &= 5.87, \quad \text{score}(\text{miña}) = 4.91, \quad \text{score}(\text{escravo}) = 4.15, \ \ldots \\
\text{weight}(\text{tía}) &= \frac{e^{\text{score}(\text{tía})}}{e^{\text{score}(\text{unha})} + e^{\text{score}(\text{tía})} + \ldots} \\
&= \frac{e^{8.48}}{e^{8.48} + e^{5.87} + e^{4.91} + e^{4.15} + \ldots} \\
&= \frac{4{,}817}{4{,}817 + 354 + 136 + 63 + \ldots} = \frac{4{,}817}{5{,}601} = 0.86
\end{aligned}
$$

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

THE ALIGNMENT PROBLEM IN TRANSLATION

*Unha **tía miña**, que todo o solucionaba con ditos e refráns, dicía algo así como "un é escravo do que di e dono do que cala".*



- The aunt is introduced as "*tía miña*" — noun before possessive, the reverse of English
- Attention learns this reordering automatically, no separate step

ATTENTION SCORES: HOW MUCH SHOULD I LOOK HERE?

- **Query**: what am I looking for? **Key**: what do I offer?
- Score $= q \cdot k$, then **softmax**: weights that sum to 1
- **Value**: the actual content — weighted average, using those weights

WORKED EXAMPLE: ALIGNING "TÍA MIÑA" → "MY AUNT"

Unha tía miña, que todo o solucionaba con ditos e refráns, dicía algo así como "un é escravo do que di e dono do que cala".

$$k_{\text{tía}} = (1, 0) \qquad k_{\text{miña}} = (0, 1)$$

To generate aunt, query $q_{\text{aunt}} = (3, 0)$:

$$\text{scores: } 3,\ 0 \quad \Rightarrow \quad \text{softmax: } 0.95,\ 0.05$$

To generate my, query $q_{\text{my}} = (0, 3)$:

$$\text{scores: } 0,\ 3 \quad \Rightarrow \quad \text{softmax: } 0.05,\ 0.95$$

- 95% attention on `tía` for aunt; 95% on `miña` for my — a clean word-level correspondence, learned from data

▶ SWITCH TO THE PAGE **Attention, step by step**

Section 3: drag the query and watch the scores, the softmax weights and the output change.

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

Activity: Attention Playground (∼15 min, individual)

- Open `01d_attention_playground.ipynb` — CPU only, no GPU needed
- Part A: rebuild this exact worked example yourself, verify the 95/5 split
- Part B: pull real attention weights out of a real translation model, on our own sentence, and see how they compare



WHAT ALIGNMENT ACTUALLY GIVES YOU

***Unha tía miña**, que todo o solucionaba con ditos e refráns, dicía algo así como "un é escravo do que di e dono do que cala".*

- **Word for word: "my aunt" —** fluent, but it assumes an aunt the reader already knows; *unha* introduces a new one

Model output: Helsinki-NLP/opus-mt-gl-en, the model from Notebook 1

WHAT ALIGNMENT ACTUALLY GIVES YOU

***Unha tía miña**, que todo o solucionaba con ditos e refráns, dicía algo así como "un é escravo do que di e dono do que cala".*

- **Word for word: "my aunt" —** fluent, but it assumes an aunt the reader already knows; *unha* introduces a new one
- **What the model writes: "An aunt of mine" —** attention keeps all three words: *unha* → an, *tía* → aunt, *miña* → of mine. Logical and faithful

Model output: Helsinki-NLP/opus-mt-gl-en, the model from Notebook 1

► SWITCH TO THE PAGE **Attention, step by step**

Section 4: where the real model looks while it writes "An aunt of mine".

WHAT ALIGNMENT ACTUALLY GIVES YOU

***Unha tía miña**, que todo o solucionaba con ditos e refráns, dicía algo así como "un é escravo do que di e dono do que cala".*

- **Word for word: "my aunt"** — fluent, but it assumes an aunt the reader already knows; *unha* introduces a new one
- **What the model writes: "An aunt of mine"** — attention keeps all three words: *unha* → an, *tía* → aunt, *miña* → of mine. Logical and faithful
- **What a translator writes: "I had an aunt who..."** — the sentence is rebuilt around introducing her; no word alignment can produce "I had"

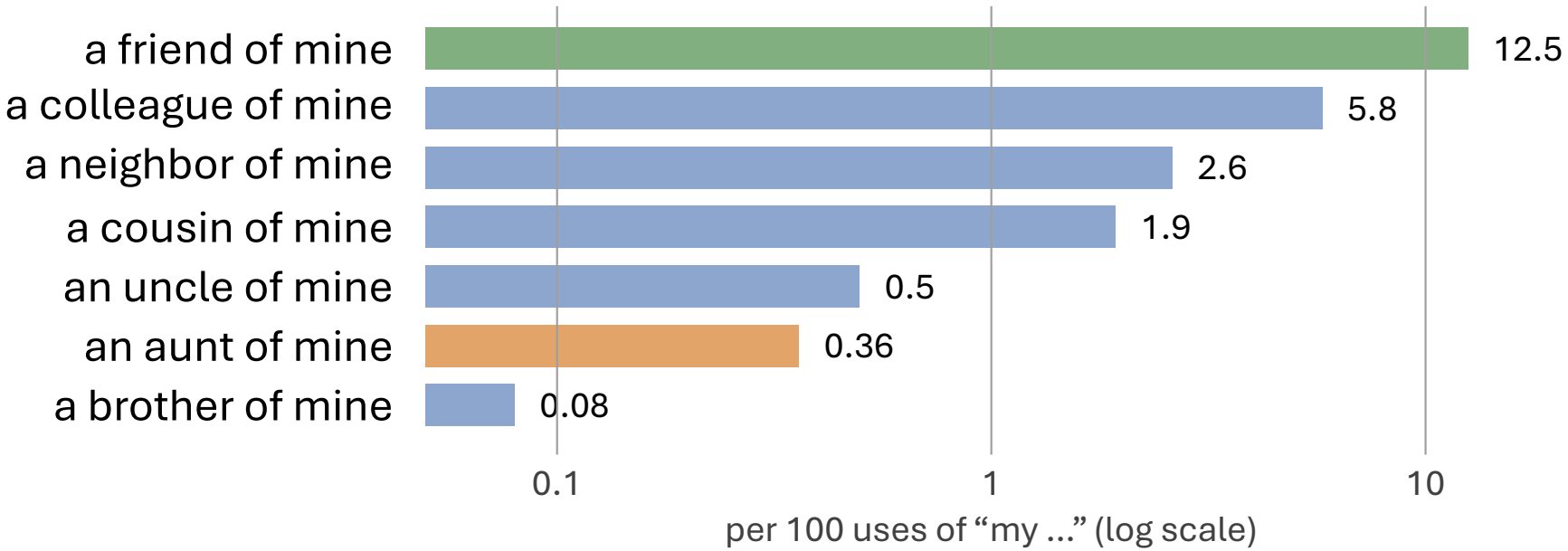
Model output: Helsinki-NLP/opus-mt-gl-en, the model from Notebook 1

► SWITCH TO THE PAGE **Neural translation, step by step**

Section 4: a toy model whose training data says "I had an aunt who". Section 5: why the real model still prefers "an aunt of mine".

HOW NATURAL IS “AN AUNT OF MINE”?

- For every 100 “my …”, how often do English books write “a … of mine”?



- “A … of mine” picks one out of several: natural for friends, marked for close family

Source: Google Books Ngram Viewer, English, 1990–2019, case-insensitive. Same counts on the n-gram page.

SELF-ATTENTION: ALIGNING A SENTENCE TO ITSELF

- Vaswani et al. (2017): the same mechanism *within* one sentence — every word attends to every word, itself included, all at once
- "It" → "the animal" — no hand-written rule needed

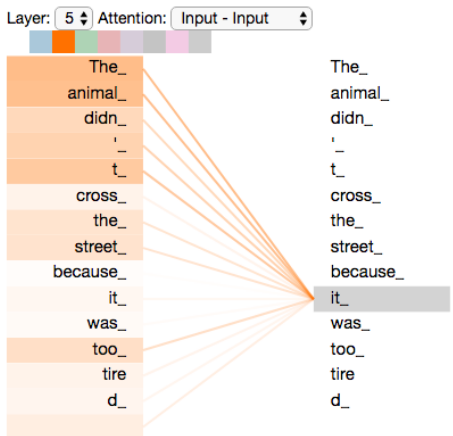
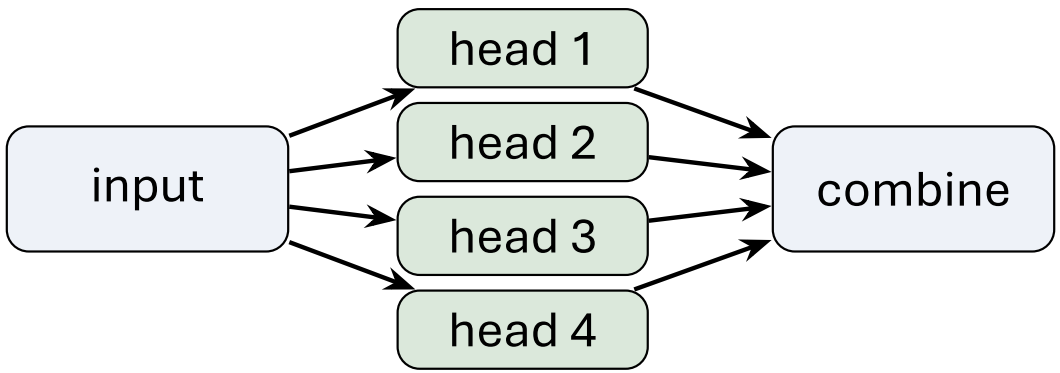


Illustration: jalammar.github.io/illustrated-transformer/ Source: Vaswani et al., NeurIPS 2017 (arXiv:1706.03762)

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

MULTIPLE ALIGNMENTS AT ONCE: MULTI-HEAD ATTENTION

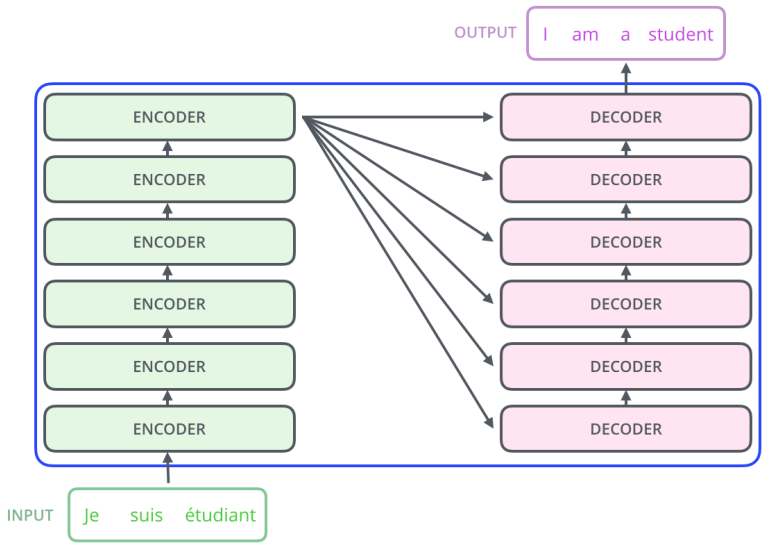
- Several attention patterns in parallel, in different “heads”, combined into one output per word
- Nobody assigns a head its job: every head starts random, and training shapes it



► SWITCH TO THE PAGE Attention, step by step
Section 5: the real model’s self-attention heads on our sentence. Section 6: who decides what a head does?

INSIDE A TRANSFORMER: STACKED BLOCKS

- Same kind of block (attention + feed-forward) repeated many times
- This picture is the classic translation shape — **encoders** on the left reading the source sentence, **decoders** on the right generating the output
- GPT-3 and most general-purpose LLMs use only the decoder side, alone — 96 blocks deep, no separate encoder at all

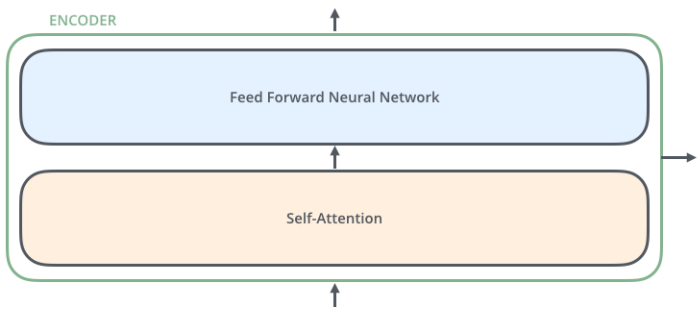


Credit: jalammar.github.io/illustrated-transformer/

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

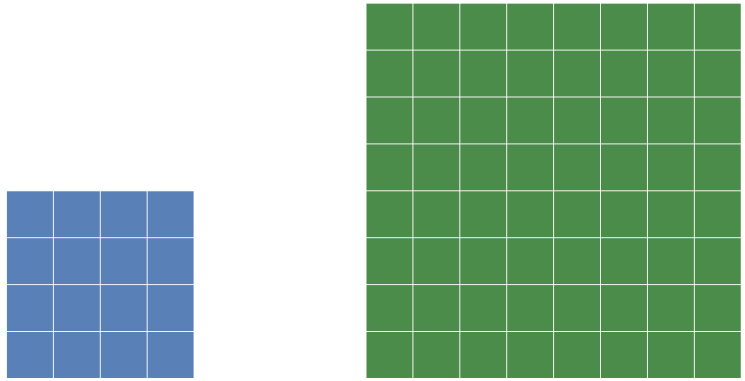
INSIDE ONE BLOCK

- Two sublayers, always in this order: **self-attention**, then a **feed-forward network**
- Self-attention mixes information across positions; the feed-forward layer does not — it runs on each word's vector separately
- This specific shape is an **encoder** block — **decoder** blocks add one more piece: **cross-attention** into the encoder's output



Credit: jalammar.github.io/illustrated-transformer/

WHY ATTENTION COST GROWS WITH CONTEXT LENGTH

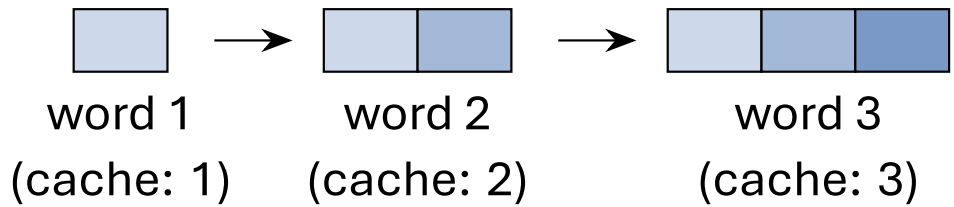


4 words
16 comparisons

8 words
64 comparisons

- Double the words → four times the comparisons — growth is quadratic
- “tía miña” was two words. The full story is thousands

GENERATING TEXT: WHY INFERENCE IS SEQUENTIAL



- One word at a time — the **KV cache** grows with every word generated
- Translating the rest of our story this way: cache growing the whole time

WHY DOES MODEL SIZE COST SO MUCH?

Challenges with RNNs	Transformers
• Long range dependencies • Gradient vanishing and explosion • Large # of training steps • Sequential/recurrence → can't parallelize • Complexity per layer: $O(n*d^2)$	• Can model long-range dependencies • No gradient vanishing and explosion • Fewer training steps • Can parallelize computation! • Complexity per layer: $O(n^2*d)$

- More parameters, more heads, more blocks → more compute per query
- Longer input → quadratically more attention cost
- **Size and context length are levers with a real, computable energy cost**

WHAT DID LLMS BRING TO MACHINE TRANSLATION?

- One model, 100+ languages — no separate model per pair
- Zero-shot, and **controllable** by instruction
- But: usually far more expensive per translation than a dedicated MT model

HOW WELL DO LLMS ACTUALLY SERVE GALICIAN?

- In mainstream LLMs, Galician's presence is "almost residual" — close enough to Spanish and Portuguese that models blur the three together
- Fix: a **smaller**, dedicated model (1.3B params), trained on real Galician text, that beat the giant general model at actually producing correct Galician
- Even bias research mostly skips languages like this: 2 papers out of dozens reviewed

Source: Gamallo et al., "A Galician-Portuguese Generative Model," EPIA 2024; Proxecto Nós (USC)

IS THERE “GREEN MT” RESEARCH?

140× faster 98.7% less energy

for a small quality trade-off

- That’s an MT-specific result — Guta, Petrick & Polák, EMNLP Findings 2025
- The 2019 “5 cars” number itself came from a machine-translation architecture search
- Quantization for MT, specifically: Shterionov & Vanmassenhove, 2022 — Session 2’s topic

THE LEVERS WE'LL ACTUALLY USE

- **Precision** (Session 2) **Retrieval** (Session 3) **Task-specificity** (today, twice over)



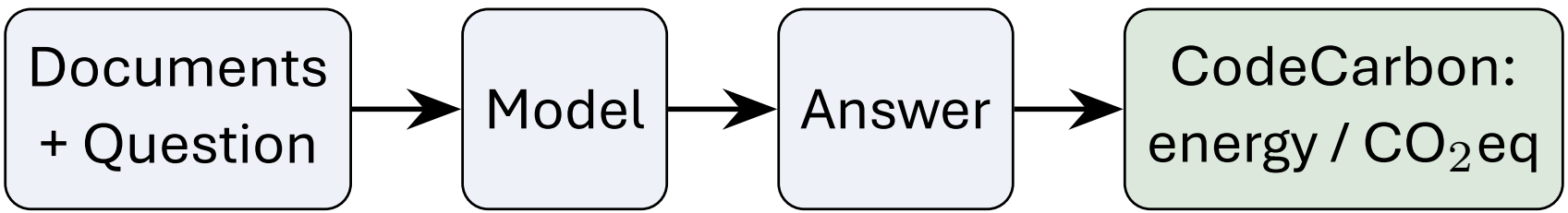
Precision Retrieval Task-specific

- Today, we build the **baseline** the other two get compared against

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

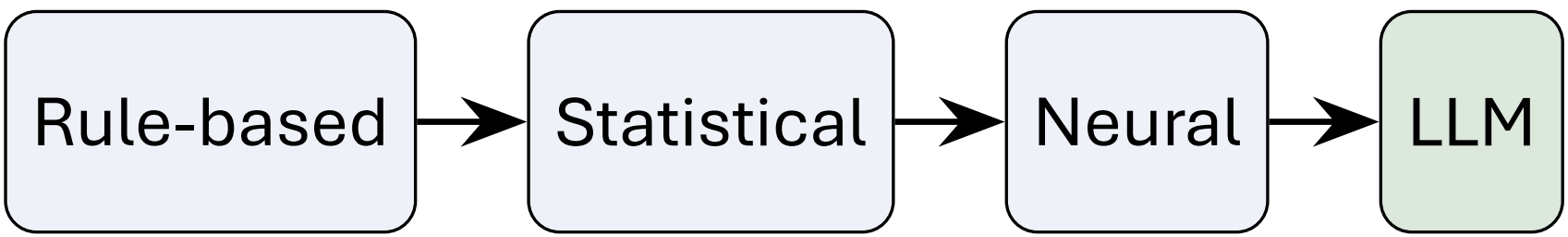
NOTEBOOK 1: BASELINE MEASUREMENT

- Task: answer questions about a small set of documents (about Green NLP itself)
- Instrumented with **CodeCarbon**: energy, CO_2eq, quality — one row on your leaderboard



NOTEBOOK 1, CONTINUED: FOUR WAYS TO TRANSLATE, THREE WAYS TO GRADE IT

- Part 2: the same sentence from this morning, through **rule-based**, **statistical**, **neural**, and **LLM** translation — the whole history, hands-on
- Part 3: **BLEU**, **BERTScore**, and an **LLM-as-judge** score the results — and we measure what grading itself costs



Activity: Notebook 1 (60–75 min baseline, +40–60 min Parts 2–3, individual)

- Open `01_baseline_measurement.ipynb` in Google Colab
- Runtime → Change runtime type → T4 GPU
- Part 1 first, then Parts 2–3 if time allows
- Stretch goal (optional): swap in a bigger model and compare



SHARE-OUT

- Part 1: how much energy/CO_2eq did your run measure? Same model, different hardware — did it matter?
- Parts 2–3: how many orders of magnitude separated your cheapest and priciest approach — and did the three MT metrics even agree?
- Next time: we quantize this exact model





GRACIAS POR SU ATENCIÓN

INDITEX UDC

Cátedra de IA en Algoritmos Verdes

https://catedrainditexalgoritmosverdes.co

BACKUP SLIDES

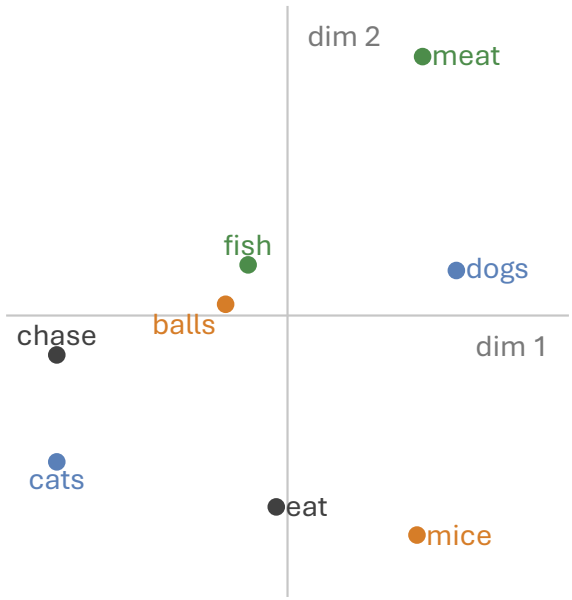
The same steps as on the pages, for when a page is not available

- Skip-gram: the table at the start, and one training step
- A refresher on multiplying matrices

THE MODEL'S PARAMETERS: A TABLE OF NUMBERS

One row per word. It starts random.

word	dim 1	dim 2
cats	−0.41	−0.26
dogs	0.30	0.08
chase	−0.41	−0.07
eat	−0.02	−0.34
mice	0.23	−0.39
balls	−0.11	0.02
fish	−0.07	0.09
meat	0.24	0.46



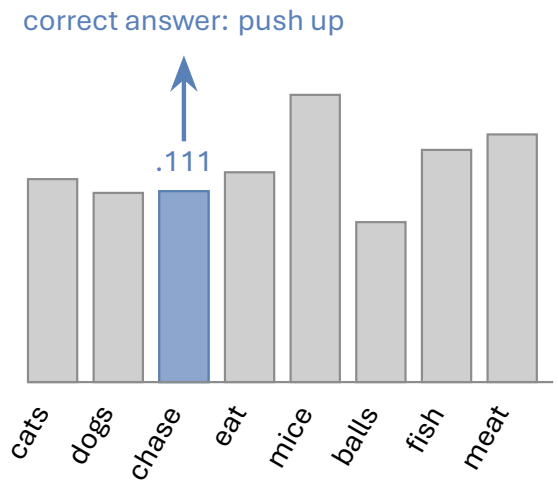
- Two dimensions so we can draw it. word2vec used 300.
- Color = same neighbors in the corpus. The model never sees the colors.

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

Backup

ONE TRAINING STEP: CATS → CHASE

Model's prediction: $P(\cdot \mid \text{cats})$



Training pair: cats → chase

Training nudges the parameters so that $P(\text{chase} \mid \text{cats})$ grows.

One of the nudged rows is *cats*:

before: $(-0.414,\ -0.263)$

after: $(-0.384,\ -0.261)$

$P(\text{chase} \mid \text{cats})$: $0.111 \rightarrow 0.115$

- Then: cats → eat, dogs → chase, dogs → eat, … all 16 pairs, thousands of times

QUICK REFRESHER: MULTIPLYING MATRICES

- To multiply A ($m \times n$) by B ($n \times p$): the **inner** dimensions must match; the result is $m \times p$
- Each entry of the result is a **dot product**: multiply matching entries, add them up

$$\begin{pmatrix} 1 & 2 & 0 \\ 3 & 1 & 4 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 1 & 3 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 4 & 6 \\ 7 & 11 \end{pmatrix}$$

top-left entry: $1 \times 2 + 2 \times 1 + 0 \times 0 = 4$

- A vector is just a matrix with one row — and a **one-hot** vector (all zeros but a single 1) does something special:

$$\begin{pmatrix} 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0.1 & 0.9 \\ 0.3 & 0.2 \\ 0.7 & 0.5 \\ 0.4 & 0.8 \end{pmatrix} = \begin{pmatrix} 0.7 & 0.5 \end{pmatrix}$$

the 1 in position 3 copies out row 3 — every other row gets multiplied by 0